

Quiz di Preparazione all'Esame

32 domande da programma ufficiale + 16 domande extra

Python · Reti · FastAPI · Bootstrap · CSS · SQL

Domande da programma ufficiale

Domande extra simili

SEZIONE 1 — Domande da programma d'esame

Python

1. Cosa si intende per type hinting in Python?

- A) Forzare i tipi a runtime impedendo errori
- B) Annotare variabili e funzioni con il tipo previsto, senza forzarlo
- C) Convertire automaticamente i tipi tra int, str e float
- D) Definire tipi personalizzati con la keyword type

2. Cosa fa un decorator in Python?

- A) Modifica il colore dell'output in console
- B) Aggiunge metadati a una classe senza cambiarla
- C) Avvolge una funzione per aggiungere comportamento prima/dopo la sua esecuzione
- D) Definisce una funzione privata non accessibile dall'esterno

3. A cosa serve la funzione abs() in Python?

- A) Restituisce il valore assoluto di un numero
- B) Arrotonda un float all'intero più vicino
- C) Restituisce la lunghezza di una stringa
- D) Converte un numero in binario

4. Cosa restituisce vars() chiamato su un oggetto?

- A) La lista dei metodi pubblici dell'oggetto
- B) Il dizionario __dict__ con gli attributi dell'istanza
- C) Il tipo dell'oggetto come stringa
- D) Il numero di attributi dell'oggetto

5. Cosa si intende per context manager in Python?

- A) Un gestore di versioni del codice
- B) Un oggetto che definisce cosa fare all'entrata e all'uscita di un blocco with
- C) Una classe che gestisce le variabili di contesto globali
- D) Un decoratore che salva il contesto di una funzione

6. Qual è la differenza principale tra threading e multiprocessing in Python?

- A) Il threading usa più CPU, il multiprocessing usa un solo core
- B) Con il threading i thread condividono la memoria; con il multiprocessing ogni processo ha la sua
- C) Il multiprocessing è più lento perché usa più memoria
- D) Non c'è differenza pratica: sono intercambiabili

7. A cosa serve asyncio in Python?

- A) Eseguire codice in parallelo su più CPU
- B) Gestire operazioni I/O asincrone con un singolo thread tramite event loop
- C) Sincronizzare i thread per evitare race conditions
- D) Creare processi figlio in background

8. Cos'è l'event loop in asyncio?

- A) Un ciclo for che itera sugli eventi di input dell'utente
- B) Il meccanismo che coordina l'esecuzione delle coroutine, decidendo quale eseguire quando un'altra è in attesa
- C) Un thread dedicato che esegue le funzioni async in parallelo
- D) Una coda FIFO di messaggi tra thread

Reti e Socket

9. Qual è il ruolo del client in un'architettura client-server?

- A) Gestire il database e rispondere alle query
- B) Inviare richieste al server e presentare le risposte all'utente
- C) Smistare il traffico di rete tra più server
- D) Autenticare gli utenti prima che accedano al server

10. Che cos'è una socket?

- A) Un protocollo di sicurezza per le connessioni HTTPS
- B) Un endpoint di comunicazione che permette a due programmi di scambiarsi dati in rete
- C) Un file speciale su disco usato per la comunicazione tra processi
- D) Un container Docker per isolare i servizi di rete

11. Qual è la differenza principale tra TCP e UDP?

- A) TCP è più veloce, UDP è più affidabile
- B) TCP garantisce la consegna ordinata e affidabile dei dati; UDP è senza connessione e non garantisce la consegna
- C) UDP cifra i dati, TCP no
- D) TCP funziona solo su rete locale, UDP su internet

FastAPI e Pydantic

12. A cosa serve Pydantic?

- A) Creare tabelle nel database tramite ORM
- B) Validare e serializzare dati usando type hints Python
- C) Gestire le sessioni utente nelle applicazioni web
- D) Generare documentazione HTML dal codice

|13. Cos'è la dependency injection in FastAPI?

- A) Un pattern per installare pacchetti Python automaticamente
- B) Un meccanismo per iniettare dipendenze (come la sessione DB) negli endpoint tramite Depends()
- C) Un modo per importare moduli esterni senza pip
- D) Una tecnica per condividere variabili globali tra router

|14. Qual è la funzione di un endpoint / route 'home' (GET /)?

- A) Restituire la documentazione dell'API
- B) Essere il punto di ingresso principale dell'API, spesso usato per health check o benvenuto
- C) Gestire il login degli utenti
- D) Redirigere tutte le richieste al database

|15. Qual è il formato di scambio dati standard tra un'API Python e un client web?

- A) XML
- B) CSV
- C) JSON
- D) HTML

Docker

|16. Cos'è Docker?

- A) Un linguaggio di programmazione per il cloud
- B) Una piattaforma per creare, distribuire ed eseguire applicazioni in container isolati
- C) Un sistema operativo leggero per server
- D) Un servizio di hosting per applicazioni Python

|17. Come avviene la virtualizzazione con i container Docker rispetto alle VM tradizionali?

- A) I container virtualizzano l'hardware completo come le VM
- B) I container condividono il kernel del sistema operativo host, rendendoli più leggeri delle VM
- C) Docker usa un hypervisor dedicato per ogni container
- D) I container Docker richiedono un sistema operativo completo per funzionare

Bootstrap e CSS

|18. Cos'è Bootstrap?

- A) Un framework JavaScript per applicazioni single-page
- B) Un framework CSS che fornisce componenti UI e un sistema a griglia responsivo
- C) Un preprocessore CSS come SASS
- D) Una libreria Python per generare CSS dinamicamente

19. In Bootstrap, cosa fa la classe 'container'?

- A) Crea un elemento con larghezza al 100% sempre
- B) Centra il contenuto e imposta una larghezza massima che varia per breakpoint
- C) Applica un padding di 15px su tutti i lati
- D) Nasconde l'elemento su dispositivi mobili

20. Cosa significa creare una barra di navigazione 'adattiva' (responsive) in Bootstrap?

- A) La navbar cambia colore in base al tema del browser
- B) La navbar mostra un menu hamburger su schermi piccoli e si espande su schermi grandi
- C) La navbar si posiziona in basso su mobile e in alto su desktop
- D) La navbar adatta automaticamente i colori agli standard di accessibilità

21. Per applicare uno stile primario a un <button> in Bootstrap cosa si deve scrivere?

- A) class="button-primary"
- B) class="btn btn-primary"
- C) style="primary"
- D) class="bootstrap-btn primary"

22. Qual è la larghezza del breakpoint 'lg' in Bootstrap 5?

- A) 768px
- B) 992px
- C) 1200px
- D) 1024px

23. Come si allineano gli elementi in colonna verticale con Flexbox?

- A) display: flex; flex-direction: row;
- B) display: flex; flex-direction: column;
- C) display: grid; columns: 1;
- D) display: flex; align-items: column;

24. Cosa sono le variabili in SASS?

- A) Classi CSS dinamiche generate a runtime
- B) Identificatori che iniziano con \$ e memorizzano valori riutilizzabili nel foglio di stile
- C) Funzioni JavaScript che modificano il CSS
- D) Attributi HTML personalizzati per lo stile

25. Qual è l'approccio 'mobile-first' corretto in CSS/Bootstrap?

- A) Scrivere prima gli stili per desktop e poi sovrascriverli per mobile con max-width
- B) Scrivere prima gli stili per mobile e poi aggiungere media query con min-width per schermi più grandi
- C) Usare JavaScript per rilevare il dispositivo e caricare il CSS corretto
- D) Creare due file CSS separati, uno per mobile e uno per desktop

26. Come si associa correttamente una <label> a un <input> in HTML?

- A) Mettendo la label e l'input nello stesso <div>
- B) Usando l'attributo for della label che corrisponde all'id dell'input
- C) Usando l'attributo name uguale su entrambi
- D) Collegandoli con JavaScript tramite addEventListener

27. Cos'è il box model CSS standard?

- A) Un modello di colori che definisce come il browser interpreta RGB e HEX
- B) Il modello che descrive ogni elemento HTML come un rettangolo composto da content, padding, border e margin
- C) Un sistema di griglia a 12 colonne per il layout
- D) Il modo in cui il browser calcola la specificità dei selettori CSS

28. Cos'è un selettore CSS?

- A) Una variabile che contiene uno stile CSS
- B) Un pattern che indica a quali elementi HTML applicare le regole di stile
- C) Un attributo HTML per applicare stili inline
- D) Una funzione JavaScript che seleziona elementi nel DOM

SQL

29. Cosa sono i comandi DDL in SQL?

- A) Comandi per interrogare i dati (SELECT, FROM, WHERE)
- B) Comandi per modificare i dati (INSERT, UPDATE, DELETE)
- C) Comandi per definire la struttura del database (CREATE, ALTER, DROP)
- D) Comandi per gestire le transazioni (COMMIT, ROLLBACK)

30. Cosa sono i comandi DML in SQL?

- A) Comandi per creare e modificare tabelle
- B) Comandi per manipolare i dati: INSERT, UPDATE, DELETE (e SELECT)
- C) Comandi per gestire gli utenti del database
- D) Comandi per ottimizzare le query con gli indici

31. Quale clausola SQL seleziona solo i record che soddisfano una condizione?

- A) HAVING
- B) ORDER BY
- C) WHERE
- D) GROUP BY

32. Quale delle seguenti è una funzione di aggregazione SQL?

- A) UPPER()
- B) COUNT()
- C) SUBSTR()
- D) COALESCE()

SEZIONE 2 — Domande extra

Python avanzato

33. Qual è la differenza tra `@staticmethod` e `@classmethod` in Python?

- A) `@staticmethod` riceve `cls`, `@classmethod` riceve `self`
- B) `@staticmethod` non riceve né `self` né `cls`; `@classmethod` riceve `cls` come primo argomento
- C) Sono identici, solo con nomi diversi
- D) `@classmethod` non può essere chiamato da un'istanza

34. Cosa fa il decorator `@property` in Python?

- A) Rende il metodo privato e non accessibile dall'esterno
- B) Permette di chiamare un metodo come se fosse un attributo (senza parentesi)
- C) Esegue il metodo automaticamente alla creazione dell'oggetto
- D) Condivide il metodo tra tutte le istanze della classe

35. In Python, qual è la differenza tra una lista e un generatore?

- A) La lista può contenere tipi misti, il generatore no
- B) La lista calcola tutti i valori subito; il generatore li produce uno alla volta (lazy)
- C) Il generatore è più veloce in scrittura ma più lento in lettura
- D) Non c'è differenza pratica

36. Cosa fa il comando `'with open("file.txt") as f:'` in Python?

- A) Apre il file in modalità scrittura e lo blocca per altri processi
- B) Apre il file e garantisce che venga chiuso automaticamente anche in caso di eccezione
- C) Crea il file se non esiste, altrimenti lo sovrascrive
- D) Legge tutto il contenuto del file in una stringa

HTTP e API

37. In HTTP, qual è la differenza tra GET e POST?

- A) GET è più sicuro di POST perché cifra i dati
- B) GET invia i dati nell'URL (query string); POST li invia nel body della richiesta
- C) POST è usato solo per aggiornare dati, GET solo per crearli
- D) Non c'è differenza: entrambi inviano dati al server

38. Cos'è un indice (INDEX) in SQL e perché si usa?

- A) Un numero progressivo assegnato automaticamente a ogni riga
- B) Una struttura dati che velocizza le query di ricerca su una o più colonne
- C) Un vincolo che garantisce l'unicità dei valori in una colonna
- D) Una vista pre-calcolata delle query più frequenti

39. Cosa fa GROUP BY in SQL?

- A) Ordina i risultati in ordine crescente
- B) Raggruppa le righe con lo stesso valore in una colonna e permette di applicare funzioni di aggregazione per gruppo
- C) Filtra i risultati dopo un JOIN
- D) Unisce due tabelle su una colonna comune

40. Quale metodo HTTP si usa convenzionalmente per aggiornare parzialmente una risorsa?

- A) PUT
- B) POST
- C) PATCH
- D) UPDATE

SQL avanzato

41. Cos'è un JOIN in SQL?

- A) Un comando per unire due database distinti
- B) Un'operazione che combina righe di due tabelle in base a una condizione (tipicamente uguaglianza di chiavi)
- C) Un modo per copiare una tabella in un'altra
- D) Un comando per unire più query in una sola

42. In Bootstrap, cosa fa la classe 'd-none d-md-block'?

- A) Nasconde l'elemento su tutti i dispositivi
- B) Mostra l'elemento solo su schermi medio-grandi, lo nasconde su schermi piccoli
- C) Applica display:block su tutti i dispositivi
- D) Nasconde l'elemento su schermi $\geq$ md

43. Cosa fa 'alembic upgrade head' in un progetto FastAPI?

- A) Crea un nuovo file di migrazione
- B) Applica tutte le migrazioni pendenti al database portandolo allo stato più recente
- C) Annulla l'ultima migrazione applicata
- D) Genera automaticamente i modelli SQLAlchemy dal database esistente

44. Cosa restituisce FastAPI automaticamente se un body JSON non corrisponde allo schema Pydantic?

- A) HTTP 400 Bad Request
- B) HTTP 500 Internal Server Error
- C) HTTP 422 Unprocessable Entity con i dettagli degli errori
- D) HTTP 404 Not Found

Bootstrap, CSS e FastAPI

45. Qual è la differenza tra INNER JOIN e LEFT JOIN?

- A) INNER JOIN è più veloce, LEFT JOIN più preciso
- B) INNER JOIN restituisce solo righe con corrispondenza in entrambe le tabelle; LEFT JOIN restituisce tutte le righe della tabella sinistra (NULL dove non c'è corrispondenza)
- C) LEFT JOIN funziona solo con chiavi primarie
- D) Non c'è differenza pratica nel risultato

46. In CSS, cosa fa 'box-sizing: border-box'?

- A) Aggiunge un bordo visibile a tutti gli elementi
- B) Fa sì che padding e border siano inclusi nella larghezza/altezza dichiarata dell'elemento
- C) Imposta il modello a scatola tridimensionale per le animazioni
- D) Disabilita il margin collapsing tra elementi adiacenti

47. Cos'è una coroutine in Python?

- A) Una funzione che può essere eseguita in parallelo su più thread
- B) Una funzione definita con `async def` che può essere sospesa con `await`
- C) Una funzione che si richiama ricorsivamente
- D) Un tipo speciale di generatore che produce coppie chiave-valore

48. Cosa fa ALTER TABLE in SQL?

- A) Elimina la tabella e la ricrea vuota
- B) Modifica la struttura di una tabella esistente (aggiunge/rimuove colonne, cambia tipi)
- C) Rinomina tutti i record della tabella
- D) Copia la struttura della tabella senza i dati

RISPOSTE CORRETTE

Sezione 1 — Domande da programma

1. **B** — Il type hinting è puramente indicativo: Python non lo verifica a runtime. Serve agli IDE, linter e librerie come Pydantic.
2. **C** — Un decorator è una funzione che riceve un'altra funzione, la avvolge (wrapper) e ne modifica il comportamento. Esempio: `@app.get('/')` in FastAPI.
3. **A** — `abs(-7)` restituisce 7. Funziona su int, float e complex.
4. **B** — `vars(obj)` equivale a `obj.__dict__`: restituisce un dict con tutti gli attributi di istanza e i loro valori.
5. **B** — I context manager implementano `__enter__` e `__exit__`. Il blocco `'with open(file) as f:'` ne è l'esempio classico: il file viene chiuso automaticamente all'uscita.
6. **B** — Threading: thread nello stesso processo, memoria condivisa, limitati dal GIL per il codice CPU-bound. Multiprocessing: processi separati, memoria separata, nessun GIL.
7. **B** — `asyncio` permette di gestire molte operazioni I/O (rete, disco) concorrentemente con un solo thread, usando `await` per sospendere l'esecuzione durante le attese.
8. **B** — L'event loop esegue una coroutine alla volta. Quando una coroutine incontra `await`, l'event loop sospende quella e ne esegue un'altra pronta, tornando alla prima quando l'attesa è terminata.

- 9. B** — Il client avvia la comunicazione inviando richieste (es. HTTP GET). Il server risponde. Il client può essere un browser, un'app mobile, o un altro programma.
- 10. B** — Una socket è identificata da indirizzo IP + porta. Rappresenta un'estremità della connessione di rete tra due programmi (anche sullo stesso host).
- 11. B** — TCP: connessione orientata, handshake, ritrasmissione in caso di perdita — usato per HTTP, FTP, email. UDP: nessuna connessione, più veloce ma inaffidabile — usato per streaming, DNS, giochi.
- 12. B** — Pydantic usa i type hints per validare automaticamente i dati in input/output. Se un campo è int e arriva una stringa non convertibile, lancia un `ValidationError`.
- 13. B** — `Depends(get_db)` dice a FastAPI di chiamare `get_db()` e passare il risultato all'endpoint. Permette di riusare logica (connessione DB, autenticazione) senza duplicare codice.
- 14. B** — La route `/` è convenzionalmente usata come health check (verificare che il server risponda) o per restituire un messaggio di benvenuto/info sull'API.
- 15. C** — JSON (JavaScript Object Notation) è il formato standard per le API REST moderne. FastAPI converte automaticamente i dict Python e gli oggetti Pydantic in JSON.
- 16. B** — Docker isola l'applicazione e le sue dipendenze in un container. 'Funziona sul mio PC' diventa 'funziona ovunque ci sia Docker'.
- 17. B** — Una VM virtualizza hardware completo + SO. Un container condivide il kernel dell'host e isola solo processi e filesystem: si avvia in secondi e occupa molto meno spazio.
- 18. B** — Bootstrap offre classi CSS predefinite (`btn`, `container`, `row`, `col-*`) e componenti pronti (`navbar`, `card`, `modal`) per costruire interfacce responsivi rapidamente.
- 19. B** — container centra orizzontalmente il contenuto e applica una `max-width` che cambia per breakpoint (`sm`, `md`, `lg`, `xl`). `container-fluid` è sempre 100%.
- 20. B** — Con `navbar-expand-lg`, la navbar mostra tutti i link su schermi $\geq$ `lg` e li collassa in un menu hamburger (toggler) su schermi più piccoli.
- 21. B** — In Bootstrap servono sempre due classi: `btn` per lo stile base del bottone e `btn-primary` (o `btn-secondary`, `btn-danger` ecc.) per il colore.
- 22. B** — I breakpoint Bootstrap 5: `sm=576px`, `md=768px`, `lg=992px`, `xl=1200px`, `xxl=1400px`. `lg` è quindi 992px.
- 23. B** — `flex-direction: column` dispone i flex item uno sotto l'altro (asse principale verticale). Di default è `row` (orizzontale).
- 24. B** — In SASS: `$colore-primario: #1F3864`; poi usato come `color: $colore-primario`. Semplifica la manutenzione: cambio il valore in un posto solo.
- 25. B** — Mobile-first: stile base = mobile. Si usa `min-width` nelle media query per aggiungere stili man mano che lo schermo si allarga. Bootstrap è progettato mobile-first.
- 26. B** — `<label for="email">Email</label><input id="email" type="email">`. Cliccando sulla label si attiva l'input (focus). Importante anche per accessibilità (screen reader).
- 27. B** — Da dentro a fuori: content (il contenuto) → padding (spazio interno) → border (bordo) → margin (spazio esterno). Con `box-sizing: border-box`, padding e border sono inclusi nella width dichiarata.
- 28. B** — Esempi: `p` (tag), `.classe`, `#id`, `p > span` (figlio diretto), `input[type='email']` (attributo). La specificità determina quale regola vince in caso di conflitto.
- 29. C** — DDL = Data Definition Language. Definisce la struttura: `CREATE TABLE`, `ALTER TABLE` (aggiunge/modifica colonne), `DROP TABLE` (elimina la tabella).
- 30. B** — DML = Data Manipulation Language. Agisce sui dati nelle tabelle. `SELECT` è tecnicamente DQL ma spesso incluso nel DML.
- 31. C** — `WHERE` filtra le righe prima di raggruppare. `HAVING` filtra dopo `GROUP BY` (sulle aggregazioni). Es: `SELECT * FROM ordini WHERE stato = 'consegnato'`;
- 32. B** — Funzioni di aggregazione: `COUNT()`, `SUM()`, `AVG()`, `MAX()`, `MIN()`. Operano su un insieme di righe e restituiscono un singolo valore. Si usano spesso con `GROUP BY`.

Sezione 2 — Domande extra

- 33. B** — `@staticmethod`: funzione normale dentro la classe, nessun accesso all'istanza. `@classmethod`: riceve `cls` e può creare istanze alternative (factory method).
- 34. B** — `@property` trasforma un metodo in un attributo leggibile. `circle.area` invece di `circle.area()`. Si abbina a `@nome.setter` per renderlo scrivibile.
- 35. B** — `[x**2 for x in range(1000)]` crea 1000 valori in memoria. `(x**2 for x in range(1000))` è un generatore: produce un valore alla volta. Più efficiente per grandi dataset.
- 36. B** — `'with'` usa il context manager di `open()`: `__enter__` apre il file, `__exit__` lo chiude. Il file viene chiuso anche se dentro il blocco viene lanciata un'eccezione.
- 37. B** — GET: dati visibili nell'URL, cacheable, idempotente — per leggere dati. POST: dati nel body, non cacheable — per creare risorse. PUT=aggiorna, DELETE=elimina.
- 38. B** — Un INDEX su una colonna permette al DB di trovare le righe molto più velocemente (come l'indice di un libro). Il downside: rallenta INSERT/UPDATE/DELETE perché l'indice va aggiornato.
- 39. B** — Es: `SELECT stato, COUNT(*) FROM ordini GROUP BY stato`; — conta gli ordini per ogni stato. `HAVING` filtra i gruppi (es: `HAVING COUNT(*) > 5`).
- 40. C** — PUT sostituisce l'intera risorsa. PATCH la aggiorna parzialmente (solo i campi inviati). FastAPI usa `@router.patch()` per questo.
- 41. B** — INNER JOIN: solo righe con corrispondenza in entrambe. LEFT JOIN: tutte le righe della sinistra + corrispondenze a destra (NULL se assente). RIGHT JOIN: l'opposto.
- 42. B** — `d-none` = `display:none` su tutti. `d-md-block` = `display:block` da md in su. L'elemento è nascosto su xs e sm, visibile da md in poi. Utilissimo per mostrare/nascondere elementi per breakpoint.
- 43. B** — `'head'` indica l'ultima migrazione nella catena. `'alembic upgrade head'` applica tutte le revisioni non ancora applicate al DB.
- 44. C** — 422 Unprocessable Entity è lo status code standard per errori di validazione. FastAPI include nel body il dettaglio di ogni campo errato (tipo atteso, valore ricevuto).
- 45. B** — Se cerchi tutti i clienti anche quelli senza ordini: `SELECT c.nome, o.data FROM clienti c LEFT JOIN ordini o ON c.id = o.cliente_id`. INNER JOIN escluderebbe i clienti senza ordini.
- 46. B** — Con `border-box: width:200px` significa che il contenuto + padding + border = 200px. Con `content-box` (default): width si riferisce solo al contenuto, padding e border si aggiungono.
- 47. B** — Le coroutine sono definite con `'async def'` e possono essere sospese con `'await'`. Non sono thread: la sospensione è cooperativa (il codice decide quando cedere il controllo).
- 48. B** — Esempi: `ALTER TABLE libri ADD COLUMN disponibile BOOLEAN DEFAULT TRUE`; — `ALTER TABLE libri DROP COLUMN vecchio_campo`; — `ALTER TABLE libri MODIFY titolo VARCHAR(500)`;